AI Query Automator

Product Documentation • v1.0

1. Executive Summary

The Wizikey AI Query Automator is an internal web tool that enables media intelligence teams to generate multilingual Boolean search queries instantly. Users enter a brand or keyword in English, select target languages, and the tool produces a ready-to-use search expression — phonetically transliterated, not meaning-translated — directly deployable on the Opoint media monitoring platform.

Key outcomes:

- Reduces query creation time from 15-20 minutes to under 10 seconds
- Eliminates dependency on language specialists for routine query building
- Supports 11 Indian languages with phonetically accurate transliteration
- One-click redirect to Opoint search with the generated query pre-filled

2. Problem Statement

Media monitoring at scale across Indian regional languages presents a recurring operational bottleneck:

- Analysts manually typed brand names in each regional language — error-prone and time-consuming
- Phonetic variations of brand names (e.g. 'Airtel' in Hindi, Bengali, Tamil) were inconsistent across teams
- No standardised process existed for multilingual query construction
- Every new campaign or brand required repeating the same manual effort

This resulted in missed coverage, inconsistent monitoring quality, and significant analyst time lost on non-analytical work.

3. Solution Overview

The AI Query Automator automates the end-to-end process of multilingual query generation:

#	Step	Description
1	User Input	Analyst enters keyword (e.g. 'Ather Energy') and selects target languages
2	Transliteration	Google Input Tools API phonetically converts keyword to each selected language
3	Query Build	Boolean query assembled with correct OR/AND operators and quoted terms
4	Review	Generated query displayed in output box for analyst to verify
5	Redirect	One click opens Opoint search with query pre-loaded — results appear instantly

3.1 Why Transliteration, Not Translation

Standard translation APIs convert meaning — 'energy' becomes 'urja' (Hindi) which is NOT how the brand appears in news articles. The tool uses Google Input Tools phonetic transliteration so 'Ather Energy' becomes '□□□ □□□□□□' — exactly as journalists write it.

3.2 Language Support

Language	Code	Notes
English	en	Casing variants: original / lowercase / UPPERCASE / Title Case
Hindi	hi	□□□□□□□ □□□□□□□□□□□□□
Bengali	bn	□□□□□□□ □□□□□□□□□□□□□□□
Gujarati	gu	□□□□□□□ □□□□□□□□□□□□□□
Marathi	mr	□□□□□□□ □□□□□□□□□□□□□□

Tamil	ta	□□□□□□□□□□□□□□□
Telugu	te	□□□□□□□□□ □□□□□□□□□□□□□□□□□□
Kannada	kn	□□□□□□□□□ □□□□□□□□□□□□□□□□□□
Malayalam	ml	□□□□□□□□□□ □□□□□□□□□□□□□□□□□□
Punjabi	pa	□□□□□□□□ □□□□□□□□□□□□□□□□
Urdu	ur	ٹرانسلیٹریشن فونیٹک

4. Technical Architecture

4.1 Tech Stack

Layer	Technology	Purpose
Backend	Python 3.x + Flask	REST API server, query generation logic
Transliteration	Google Input Tools API	Free, no API key required, phonetic accuracy
Frontend	HTML5 + CSS + JavaScript	Lightweight, no framework dependency
Styling	Custom CSS with CSS Variables	Dark theme, responsive, animated UI
Deployment	Flask dev server	Single command: python app.py

4.2 Project Structure

```
QUERY_AUTOMATOR/
├── app.py              ← Flask backend + transliteration + query builder
├── static/
|   ├── style.css       ← Main stylesheet (dark theme)
|   ├── script.js       ← Pill selection, copy, Opoint redirect
|   └── wizikey-logo-1.png
└── templates/
    └── index.html      ← Main UI
```

4.3 API Endpoints

Method	Endpoint	Description
GET	/	Serves the main UI (index.html)
POST	/generate	Accepts keyword + languages + operator, returns Boolean query JSON
GET	/health	Health check — confirms server is running

4.4 POST /generate — Request & Response

Request Body (JSON)

```
{
  "keyword":   "Ather Energy",
  "operator":  "AND",
  "languages": ["English", "Hindi", "Bengali"]
}
```

Response Body (JSON)

```
{
  "query": "(\"Ather Energy\" OR \"ather energy\" OR \"ATHER ENERGY\" OR \"□□□ □□□□□□□\" OR \"□□□□ □□□□□□□□\")",
  "raw_query": "(\"Ather Energy\" OR \"ather energy\" OR \"ATHER ENERGY\" OR \"□□□ □□□□□□□\" OR \"□□□□ □□□□□□□□\")"
}
```

4.5 Transliteration Logic

Each word in the keyword is individually sent to the Google Input Tools API, phonetically converted, then joined back together. This ensures accurate conversion for multi-word brand names.

- API: inputtools.google.com/request
- Parameter: itc={lang_code}-t-i0-und (phonetic input method code)
- Results are cached in-memory per session to avoid duplicate API calls
- On failure, original English word is used as fallback

4.6 English Casing Variants

Variant	Example	Method
User types	Ather Energy	Preserved as-is
Lowercase	ather energy	keyword.lower()
Uppercase	ATHER ENERGY	keyword.upper()
Title Case	Ather Energy	keyword.title()

5. Frontend Design

5.1 UI Components

- Logo — Wizikey branding, resizable via .logo-image CSS class
- Keyword input — text field, comma-separated for multiple keywords

- Operator select — AND / OR dropdown
- Language pills — horizontal clickable pills, toggle on/off, all 11 languages
- Generate Query button — triggers POST /generate, shows loading state
- Output textarea — readonly, displays generated query
- Copy button — copies query to clipboard, shows confirmation toast
- Open in Opoint button — URL-encodes query and opens Opoint in new tab

5.2 Opoint Redirect URL Format

`https://m360.opoint.com/search/?expression={URL_ENCODED_QUERY}&filters=geo:1144`

The generated query is URL-encoded and inserted as the expression parameter. The geo:1144 filter is fixed for India-specific results. User only needs to press Enter on Opoint to fetch results.

6. Limitations & Future Scope

6.1 Current Limitations

Limitation	Detail
Transliteration accuracy	Google Input Tools may not capture all brand variants — human review recommended for critical queries
No authentication	Currently open access — suitable for internal use only
In-memory cache	Translation cache resets on server restart — no persistence
Fixed geo filter	Opoint redirect is hardcoded to geo:1144 (India)

6.2 Planned Improvements

- Word-level control — choose which words to transliterate vs keep in English
- Saved query templates — persist frequently used queries
- Multi-geo support — allow selecting different Opoint geo filters
- Bulk keyword input — upload CSV, generate queries in batch
- Query history — log of previously generated queries per session
- Auth integration — SSO with Wizikey internal systems

7. Installation & Setup

7.1 Prerequisites

- Python 3.8 or higher
- pip package manager
- Internet connection (for Google Input Tools API)

7.2 Setup Steps

Step	Action	Command / Notes
1	Copy project	Place QUERY_AUTOMATOR folder on the server
2	Install packages	pip install flask requests
3	Run server	python app.py
4	Access tool	Open http://localhost:5000 in browser

7.3 Dependencies

Package	Purpose
flask	Web framework — serves UI and API endpoints
requests	HTTP client — calls Google Input Tools API

No paid APIs. No environment variables. No .env file required.

8. Security Considerations

- No user data stored — all processing is stateless per request
- No API keys in source code — Google Input Tools API is public and keyless
- Tool should be deployed behind internal network / VPN for production
- Flask debug=True should be disabled in production deployment

9. Quick Reference

Start the Tool

```
python app.py
```

Example Output

Keyword: "Airtel" | Languages: English, Hindi, Tamil

```
Result: ("Airtel" OR "airtel" OR "AIRTEL" OR "□□□□□□□" OR "□□□□□□□□")
```

Support

For issues or feature requests, contact the Wizikey engineering team.